

Additional Requirements

Frontend UI Framework

The frontend must use modern UI components based on **shadcn/ui**.

Technology Stack

- React 19
- TypeScript
- Vite
- shadcn/ui
- Tailwind CSS
- React Router
- React Hook Form
- Zod
- TanStack Query
- Axios
- Lucide Icons
- Framer Motion (where appropriate)
- Recharts for dashboards

UI Expectations

The application should have an enterprise look and feel similar to modern SaaS products.

Include:

- Responsive layout
- Sidebar navigation
- Top navigation bar
- Breadcrumbs
- Theme support (Light/Dark)
- Dashboard cards
- Modern data tables
- Searchable dropdowns
- Multi-select controls
- Loading skeletons
- Toast notifications
- Modal dialogs
- Stepper/Wizard for Origin Calculation
- Reusable components
- Proper form validation
- Accessibility (WCAG friendly)

The UI should be component-driven and reusable.

Generic Rule Engine (Very Important)

The Rule Engine must NOT contain hard-coded business rules.

Instead, it should be designed as a configurable, metadata-driven Business Rules Engine.

New rules should be created, modified, enabled, disabled, reordered, or deleted by administrators from the frontend without requiring code changes or application deployment.

Rule Engine Architecture

Use a combination of:

- Strategy Pattern
- Specification Pattern
- Factory Pattern
- Chain of Responsibility
- Rule Pipeline

Every rule should implement a common interface.

Example

```
public interface IRule
{
    Task<RuleResult> ExecuteAsync(RuleContext context);
}
```

Rules should be discovered and executed dynamically.

The Rule Engine should only orchestrate rule execution and should not contain business-specific logic.

Dynamic Rule Configuration

Rules should be stored in PostgreSQL rather than in code.

Each rule should contain metadata such as:

- Rule Name
- Rule Code

- Rule Category
 - Rule Type
 - Description
 - Expression / Condition
 - Parameters (JSON)
 - Priority
 - Execution Order
 - Error Message
 - Success Message
 - Effective Date
 - Expiry Date
 - Agreement
 - Country
 - HS Code
 - Enabled
 - Version
 - Created By
 - Modified By
-

Rule Execution Flow

The Rule Engine should:

1. Load applicable rules from the database.
 2. Filter rules based on the selected agreement, country, HS code, and product.
 3. Sort rules by execution order and priority.
 4. Execute each rule sequentially or according to configured dependencies.
 5. Store intermediate results in a shared execution context.
 6. Support short-circuit evaluation when configured.
 7. Produce a detailed execution log.
 8. Return a complete evaluation report.
-

Rule Builder (Frontend)

Provide an administrator interface to manage rules.

Capabilities:

- Create new rules
- Edit existing rules
- Clone rules
- Enable/Disable rules
- Reorder execution

- Version rules
 - Activate future versions
 - View execution history
 - Test a rule with sample input before publishing
-

Rule Expressions

Where possible, avoid compiling C# code for each rule.

Instead, support configurable expressions or JSON-based conditions.

Examples:

- Value Added > 60%
- Non-Originating Cost <= 40%
- HS Chapter Changed
- Country == "Germany"
- ManufacturingOperation == "Spinning"

The design should allow replacing the expression evaluator in the future without affecting the application.

Extensibility

The Rule Engine must be plugin-friendly.

Adding a new rule type should only require:

1. Creating a new rule evaluator.
2. Registering it via Dependency Injection.
3. Configuring it through the UI.

No existing business logic should need modification.

Rule Execution History

Store every calculation execution.

Include:

- Calculation ID
- Product

- Agreement
 - User
 - Date
 - Rules Executed
 - Rule Results
 - Pass/Fail
 - Execution Time
 - Input Data
 - Output Data
 - Decision Reason
-

Rule Visualization

Provide a frontend screen to visualize rule execution.

Display:

- Rules executed
- Execution order
- Passed rules
- Failed rules
- Skipped rules
- Time taken
- Final decision

This should resemble a workflow or decision tree, making it easy for users to understand how the final origin determination was reached.

Future-Proof Design

The entire solution should be designed so that additional trade agreements or rule sets can be introduced without changing existing application code.

The architecture should support:

- New countries
- New agreements
- New rule categories
- New calculation methods
- New product-specific rules
- Country-specific overrides
- Agreement-specific overrides
- Versioned rule sets

- Localization
- Multi-language support
- Multi-tenant deployment

The system should prioritize configuration over code wherever feasible, ensuring long-term maintainability and flexibility.